Core hooks

16A. Core Hooks

Questions: 7 **Difficulty:** Junior to Mid **Companies:** Every company

Q 16.1 What are the gotchas with useState? Explain lazy initialization, batching, and functional updates.

Difficulty: Junior **Asked at:** Every company

Lazy initialization — avoid expensive computation on every render:

JSX Copy

```jsx
// BAD: computeInitialState() runs on EVERY render, not just the first
const [state, setState] = useState(computeExpensiveInitialState());

// GOOD: Pass a function — only runs on the first render
const [state, setState] = useState(() => computeExpensiveInitialState());
```

The function form (`() => value`) is only called during the initial render. The direct form (`value`) is evaluated on every render even though the result is thrown away after the first.

Functional updates — avoid stale state in closures:

JSX Copy

```
  const [count, setCount] = useState(0);

  function handleClick() {
    setCount(count + 1);  // 'count' is captured from this render's closur
    setCount(count + 1);  // Same 'count' — both set to the same value
    // Result: count increases by 1, not 2
  }
}

// FIX: Use functional update — always gets the latest state
function Counter() {
  const [count, setCount] = useState(0);

  function handleClick() {
    setCount(prev => prev + 1);  // prev is always current
    setCount(prev => prev + 1);  // prev is now prev+1
    // Result: count increases by 2
  }
}
```

Batching — multiple setState calls = one re-render:

React 18 batches all state updates (even inside promises, setTimeout, and event handlers) into a single re-render. This is automatic — you don't need to do anything.

JSX Copy

```
function handleClick() {
  setName('Ankit');     // Doesn't re-render yet
  setAge(28);           // Doesn't re-render yet
  setLoading(false);    // Doesn't re-render yet
  // React batches all three → ONE re-render
}

// Even in async code (React 18+)
async function fetchData() {
  const data = await api.getData();
  setData(data);        // Batched
```

Objects and arrays — state is immutable:

JSX Copy

```jsx
// BUG: Mutating state directly — React doesn't detect the change
const [user, setUser] = useState({ name: 'Ankit', age: 28 });
user.age = 29;        // WRONG — mutating existing object
setUser(user);        // React sees same reference → no re-render

// FIX: Create a new object
setUser({ ...user, age: 29 });        // New object → re-render
setUser(prev => ({ ...prev, age: 29 })); // Functional update version
```

Q 16.2 What are the common pitfalls of useEffect? Explain dependency arrays, cleanup, and race conditions.

Difficulty: Mid **Asked at:** Every company

Missing dependencies — the stale closure bug:

JSX Copy

```jsx
// BUG: count is stale — always logs the initial value
useEffect(() => {
  const id = setInterval(() => {
    console.log(count);  // Always 0 — captured from initial render
  }, 1000);
  return () => clearInterval(id);
}, []);  // Empty deps → effect never re-runs, never gets new 'count'

// FIX: Include count in dependencies
useEffect(() => {
  const id = setInterval(() => {
    console.log(count);  // Always current
```

```
// BETTER FIX: Use functional setState to avoid the dependency entirely
useEffect(() => {
  const id = setInterval(() => {
    setCount(prev => prev + 1);  // No dependency on count
  }, 1000);
  return () => clearInterval(id);
}, []);  // Truly stable — runs once
```

Cleanup — preventing memory leaks and stale updates:

JSX Copy

```
useEffect(() => {
  const controller = new AbortController();

  fetch(`/api/users/${userId}`, { signal: controller.signal })
    .then(r => r.json())
    .then(setUser)
    .catch(err => {
      if (err.name !== 'AbortError') setError(err);
    });

  // Cleanup: cancel the request if userId changes before response arrives
  return () => controller.abort();
}, [userId]);
```

Without cleanup, if `userId` changes from "1" to "2" quickly, the response for "1" might arrive after "2" and overwrite the correct data with stale data.

Race conditions with async effects:

JSX Copy

```
// BUG: Race condition — stale response overwrites fresh one
useEffect(() => {
  fetchUser(userId).then(setUser);
  // If userId changes: 1→2, response for 1 might arrive after response fo
}, [userId]);
```

```
  let cancelled = false;

  fetchUser(userId).then(data => {
    if (!cancelled) setUser(data);  // Only update if this effect is still
  });

  return () => { cancelled = true; };
}, [userId]);
```

When NOT to use useEffect:

- Transforming data for rendering → use `useMemo` instead
- Handling user events → use event handlers, not effects
- Initializing state from props → use lazy initialization in `useState`
- Subscribing to external stores → use `useSyncExternalStore`

Q 16.3 What are the performance problems with useContext?

Difficulty: Mid **Asked at:** Google, Flipkart, Razorpay

Covered in depth in Chapter 10 (Q10.2). Summary: every consumer re-renders when the context value changes, even if they only use a small piece of it. Fixes: split contexts by change frequency, memoize the provider value, and use a state management library (Zustand) when context performance becomes a problem.

Q 16.4 When do you use useReducer over useState?

Difficulty: Mid **Asked at:** Amazon, Flipkart, Razorpay, Atlassian

Use `useReducer` when state updates depend on the previous state AND the logic is complex enough that multiple `setState` calls become confusing.

JSX

Copy

```jsx
// useState approach — getting messy with complex state
function useForm() {
  const [values, setValues] = useState({});
  const [errors, setErrors] = useState({});
  const [touched, setTouched] = useState({});
  const [isSubmitting, setIsSubmitting] = useState(false);
  // 4 separate setState calls for one logical operation (submit)
}

// useReducer approach — all state transitions in one place
const initialState = { values: {}, errors: {}, touched: {}, isSubmitting:

function formReducer(state, action) {
  switch (action.type) {
    case 'SET_FIELD':
      return {
        ...state,
        values: { ...state.values, [action.field]: action.value },
        touched: { ...state.touched, [action.field]: true },
      };
    case 'SET_ERRORS':
      return { ...state, errors: action.errors };
    case 'SUBMIT_START':
      return { ...state, isSubmitting: true, errors: {} };
    case 'SUBMIT_SUCCESS':
      return { ...initialState };
    case 'SUBMIT_FAILURE':
      return { ...state, isSubmitting: false, errors: action.errors };
    default:
      return state;
  }
}

function useForm() {
  const [state, dispatch] = useReducer(formReducer, initialState);
  // One dispatch per action — all related state changes happen together
}
```

component. Easier to debug (log every action to see state changes).

When useState is simpler: Independent pieces of state (a toggle, an input value, a counter). If your state is a single primitive or a simple object with independent fields, useReducer adds unnecessary structure.

Q 16.5 When does useCallback actually help? What's the common misuse?

Difficulty: Mid **Asked at:** Google, Amazon, Flipkart, Razorpay

`useCallback` memoizes a function so it keeps the same reference across renders. It's useful in exactly one scenario: when passing a function to a memoized child component.

JSX Copy

```jsx
// useCallback is USEFUL here — prevents ExpensiveList from re-rendering
const handleItemClick = useCallback((id) => {
  setSelectedId(id);
}, []);

return <ExpensiveList items={items} onItemClick={handleItemClick} />;
// ExpensiveList is React.memo'd — stable onItemClick reference means
// it only re-renders when 'items' changes
```

JSX Copy

```jsx
// useCallback is USELESS here — the child isn't memoized
const handleClick = useCallback(() => {
  setCount(c => c + 1);
}, []);

return <button onClick={handleClick}>Click</button>;
// <button> re-renders when its parent re-renders regardless
// useCallback just adds overhead for no benefit
```

memoized. You're paying the cost of memoization (dependency comparison) without the benefit.

When it matters beyond memo:

- Dependencies of other hooks (`useEffect` with a function dependency)
- Functions passed to custom hooks that use referential equality
- Functions stored in refs for event handlers

Q 16.6 When should you use useMemo? When does it hurt more than help?

Difficulty: Mid **Asked at:** Google, Amazon, Flipkart

`useMemo` caches the result of an expensive computation and only recalculates when dependencies change.

JSX Copy

```jsx
// USEFUL: Expensive filtering/sorting that shouldn't run every render
const filteredProducts = useMemo(() => {
  return products
    .filter(p => p.category === selectedCategory)
    .sort((a, b) => a.price - b.price);
}, [products, selectedCategory]);
```

JSX Copy

```jsx
// WASTEFUL: Simple computation that's faster to re-compute than to memoiz
const fullName = useMemo(() => {
  return `${firstName} ${lastName}`;
}, [firstName, lastName]);
// String concatenation takes nanoseconds. Memoization overhead > computat
```

memoized children (referential equality).

When useMemo hurts: Simple calculations, small data sets, objects that are recreated every render regardless (inside non-memoized components), and premature optimization before measuring.

The guideline: Profile first. If a computation takes <1ms, memoizing it adds complexity without measurable benefit. If it takes >10ms and runs on every render, `useMemo` is justified.

Q 16.7 What are the use cases for useRef beyond DOM access?

Difficulty: Mid **Asked at:** Amazon, Flipkart, Atlassian, Swiggy

Most people know `useRef` for DOM element access. But its core purpose is holding a mutable value that persists across renders without causing re-renders.

DOM access (the obvious use):

JSX Copy

```jsx
const inputRef = useRef(null);
const focusInput = () => inputRef.current?.focus();
return <input ref={inputRef} />;
```

Storing the previous value:

JSX Copy

```jsx
function usePrevious(value) {
  const ref = useRef();
  useEffect(() => {
    ref.current = value;
  });
  return ref.current;  // Returns the previous render's value
```

```
// On render where count = 5, prevCount = 4
```

Instance variables (mutable state that doesn't trigger re-render):

JSX Copy

```jsx
function Timer() {
  const intervalRef = useRef(null);

  function start() {
    intervalRef.current = setInterval(() => { /* ... */ }, 1000);
  }

  function stop() {
    clearInterval(intervalRef.current);
  }
  // Storing the interval ID in ref — changing it doesn't re-render
}
```

Tracking if component is mounted (for async operations):

JSX Copy

```jsx
function useIsMounted() {
  const mounted = useRef(false);
  useEffect(() => {
    mounted.current = true;
    return () => { mounted.current = false; };
  }, []);
  return mounted;
}

// In an async function
const isMounted = useIsMounted();
const data = await fetchData();
if (isMounted.current) setData(data);  // Only update if still mounted
```

survive re-renders. The difference is reactivity.

QUICK CHECK — no marks, just for you.

1. You write useState(computeExpensiveInitialState()) instead of useState(() => computeExpensiveInitialState()). What actually happens?

- computeExpensiveInitialState() runs only on the first render; the two forms are equivalent
- computeExpensiveInitialState() runs on every render and re-initializes the state each time, resetting it
- computeExpensiveInitialState() runs on every render, and its result is discarded after the first render
- React throws an error because a non-function initial value must be a primitive

Check answer

2. A developer wraps every inline handler in useCallback, including one passed straight to a plain <button onClick={...}>. According to the lesson, what is the effect of that particular useCallback?

- It prevents the button from re-rendering when the parent re-renders
- It causes a stale closure bug, because the handler captures the first render's state forever
- It is wasted work: the button re-renders with its parent anyway, so you pay the memoization cost with no benefit
- It is required, otherwise React recreates the DOM button node on every render

3. An effect runs fetchUser(userId).then(setUser) with [userId] as dependencies and no cleanup. The user quickly navigates from userId 1 to userId 2. What is the bug the lesson describes?

- ○ Both requests are batched by React 18 into a single request, so only user 1 is fetched
- ○ React throws an error because an async effect must return a cleanup function
- ○ The response for userId 1 can arrive after the response for userId 2 and overwrite the correct data with stale data
- ○ The effect never re-runs for userId 2, so the UI keeps showing user 1 forever

Check answer

‹ Previous
Reasoning in the UI

Next ›
Specialized hooks

LEARN MORE

ARTICLES & DOCS

Built-in React hooks (reference)
react.dev

VIDEOS



8 React hooks explained
YouTube



Master useState, useEffect & more

Frontend System Desi...



All React hooks in 12 minutes

YouTube

Hand-picked references for this lesson.